

CoWorker v3 — Frontend Integration Spec

Purpose: This document describes every API endpoint, data shape, and WebSocket event the frontend expects from the backend. Replace the mock data in `src/data/mock.ts` with real API calls to these endpoints and the UI will work end-to-end.

Base URL: `https://coworker.mcands.com.au/api/v1`

WebSocket: `wss://coworker.mcands.com.au/ws/{user_id}`

Auth: Bearer token in `Authorization` header (JWT). All endpoints require auth unless noted.

Table of Contents

1. [Authentication](#)
2. [Dashboard](#)
3. [Approvals](#)
4. [Plugins](#)
5. [Memory](#)
6. [Knowledge Graph](#)
7. [Activity](#)
8. [Findings](#)
9. [Chat](#)
10. [Specialists](#)
11. [Settings](#)
12. [WebSocket Events](#)
13. [Shared Types](#)

1. Authentication

POST /auth/login

No auth required.

Request body:

```
{
  "email": "eliza@mcands.com.au",
  "password": "string"
}
```

Response 200 :

```
{
  "access_token": "eyJ...",
  "token_type": "bearer",
  "user": {
    "id": "u1",
    "name": "Eliza Marchetti",
    "initials": "EM",
    "role": "Senior Accountant",
    "email": "eliza@mcands.com.au",
    "firm": "MC & S Accountants"
  }
}
```

The frontend stores `access_token` in `localStorage` and attaches it as `Authorization: Bearer <token>` on every subsequent request.

2. Dashboard

GET /dashboard/summary

Returns all data needed to render the Dashboard page in a single call.

Response 200 :

```
{
  "greeting_name": "Eliza",
  "date_label": "Friday, 23 May 2026",
  "pending_approvals": 7,
  "actions_today": 11,
  "findings_count": 4,
  "token_usage": {
    "used": 18245,
    "cap": 50000
  },
  "scheduled_today": [
    {
      "time": "06:30",
      "plugin": "Morning Briefing",
      "status": "done"
    },
    {
      "time": "09:00",
      "plugin": "BAS Reminder (T-7)",
      "status": "done"
    },
    {
      "time": "15:00",
      "plugin": "Follow-up Tracker",
      "status": "upcoming"
    },
    {
      "time": "17:00",
      "plugin": "Proactive Intelligence",
      "status": "upcoming"
    }
  ],
  "recent_activity": [
    {
      "id": "tr1",
      "time": "14:23",
      "plugin": "Smart Responder",
      "client": "Acme Pty Ltd",
      "action": "Drafted reply to \"Re: BAS query Q3 2026\"",
      "status": "queued",
      "tokens": 1842,
      "duration": "15.6s"
    }
  ],
  "high_priority_findings": [
```

```
{
  "id": "f3",
  "title": "Acme's Q3 BAS data is 3 days late vs typical pattern",
  "severity": "high"
}
```

`scheduled_today[].status` values: `"done"` | `"upcoming"` | `"running"`

3. Approvals

GET /approvals

Returns the full pending approvals queue for the authenticated user.

Query params:

Param	Type	Description
<code>status</code>	<code>string</code>	Filter by status. Default: <code>pending</code> . Values: <code>pending</code> , <code>approved</code> , <code>rejected</code>
<code>limit</code>	<code>int</code>	Max results. Default: <code>50</code>
<code>offset</code>	<code>int</code>	Pagination offset. Default: <code>0</code>

Response 200 :

```

{
  "total": 7,
  "items": [
    {
      "id": "a1",
      "client": "Acme Pty Ltd",
      "plugin": "Smart Responder",
      "subject": "Re: BAS query Q3 2026",
      "confidence": 0.78,
      "sla_remaining": "2h 14m",
      "sla_risk": "medium",
      "category": "email_draft",
      "draft": "Hi Sarah,\n\nThank you for sending through...",
      "reasoning": [
        {
          "step": "Looked up Acme Pty Ltd in XPM",
          "duration": "0.3s"
        }
      ],
      "lessons_applied": [
        "Acme prefers \"Hi Sarah\" not \"Dear Sarah\" (P9)"
      ],
      "why_queued": "Confidence below firm threshold (0.85)...",
      "created_at": "2026-05-23T14:23:00+10:00",
      "status": "pending"
    }
  ]
}

```

category values: "email_draft" | "document_action"

sla_risk values: "low" | "medium" | "high"

POST /approvals/{id}/approve

Approve an item as-is or with an edited draft.

Request body:

```
{
  "edited_draft": "Hi Sarah,\n\n[optional edited content]"
}
```

If `edited_draft` is omitted, the original draft is used.

Response 200 :

```
{ "id": "a1", "status": "approved", "sent_at": "2026-05-23T14:31:00+10:00" }
```

POST /approvals/{id}/reject

Reject an approval item.

Request body:

```
{
  "reason": "optional rejection note"
}
```

Response 200 :

```
{ "id": "a1", "status": "rejected" }
```

4. Plugins

GET /plugins

Returns all plugins with their current state.

Response 200 :

```
{
  "items": [
    {
      "id": "p1",
      "name": "Smart Responder",
      "description": "Drafts replies to incoming client emails using full
context.",
      "trigger": "On email arrival",
      "enabled": true,
      "runs_today": 12,
      "last_run_at": "2026-05-23T14:23:00+10:00",
      "last_run_display": "14:23",
      "status": "ok"
    }
  ]
}
```

status values: "ok" | "running" | "error" | "disabled"

PATCH /plugins/{id}

Enable or disable a plugin.

Request body:

```
{ "enabled": true }
```

Response 200 :

```
{ "id": "p1", "enabled": true }
```

POST /plugins/{id}/run

Trigger a plugin to run immediately (manual trigger).

Response 202 :

```
{ "id": "p1", "run_id": "run_abc123", "status": "running" }
```

5. Memory

GET /memory

Returns the paginated memory store (interactions, lessons, documents).

Query params:

Param	Type	Description
q	string	Full-text search across client and summary
type	string	Filter: <code>interaction</code> , <code>lesson</code> , <code>document</code> . Omit for all
client	string	Filter by client name
limit	int	Default: 50
offset	int	Default: 0

Response 200 :

```

{
  "total": 8,
  "items": [
    {
      "id": "m1",
      "type": "interaction",
      "client": "Acme Pty Ltd",
      "summary": "Discussed Q3 BAS figures...",
      "date": "2026-05-20",
      "source": "Smart Responder",
      "priority": null
    },
    {
      "id": "m2",
      "type": "lesson",
      "client": "Acme Pty Ltd",
      "summary": "Always quote BAS lodgement deadlines in correspondence.",
      "date": "2026-04-15",
      "source": "Reflection",
      "priority": 8
    }
  ]
}

```

type values: "interaction" | "lesson" | "document"

priority is null for non-lessons; 1–10 integer for lessons (higher = more important).

6. Knowledge Graph

GET /knowledge-graph/nodes

Returns all graph nodes for the authenticated user's client base.

Query params:

Param	Type	Description
client_id	string	Filter to a specific client's entity family

Response 200 :

```
{
  "nodes": [
    {
      "id": "n1",
      "label": "Acme Pty Ltd",
      "type": "primary_entity",
      "x": 400,
      "y": 250
    },
    {
      "id": "n2",
      "label": "Sarah Chen (Director)",
      "type": "person",
      "x": 200,
      "y": 100
    }
  ],
  "edges": [
    {
      "id": "e1",
      "source": "n1",
      "target": "n2",
      "label": "director"
    }
  ]
}
```

node.type values: "primary_entity" | "related_entity" | "person" | "cross_client" | "other"

The frontend maps these to colours:

- primary_entity → #142234 (navy)
 - related_entity → #3080bc (cobalt)
 - cross_client → #eb881f (orange)
 - person / other → #858481 (muted)
-

7. Activity

GET /activity

Returns the chronological agent trace log.

Query params:

Param	Type	Description
date	string	ISO date, e.g. 2026-05-23 . Default: today
plugin	string	Filter by plugin name
status	string	Filter by status
limit	int	Default: 100
offset	int	Default: 0

Response 200 :

```
{
  "total": 11,
  "items": [
    {
      "id": "tr1",
      "time_display": "14:23",
      "timestamp": "2026-05-23T14:23:00+10:00",
      "plugin": "Smart Responder",
      "client": "Acme Pty Ltd",
      "action": "Drafted reply to \"Re: BAS query Q3 2026\"",
      "status": "queued",
      "tokens": 1842,
      "duration": "15.6s",
      "trace_detail": {
        "steps": [
          { "step": "Looked up Acme Pty Ltd in XPM", "duration": "0.3s" }
        ],
        "model": "claude-3-5-sonnet-20241022",
        "total_tokens": 1842,
        "prompt_tokens": 1240,
        "completion_tokens": 602
      }
    }
  ]
}
```

status values: "queued" | "approved" | "sent" | "rejected" | "running" | "error"

8. Findings

GET /findings

Returns proactive intelligence findings.

Query params:

Param	Type	Description
severity	string	Filter: high , medium , info
dismissed	bool	Include dismissed findings. Default: false

Response 200 :

```
{
  "items": [
    {
      "id": "f1",
      "severity": "high",
      "category": "DEADLINE RISK",
      "title": "Acme's Q3 BAS data is 3 days late vs typical pattern",
      "detail": "Last 8 quarters, Acme has sent BAS figures by day 6...",
      "suggested_action": "Send urgent nudge",
      "created_at": "2026-05-23T07:00:00+10:00",
      "dismissed": false
    }
  ]
}
```

severity values: "high" | "medium" | "info"

POST /findings/{id}/act

Trigger the suggested action for a finding (e.g. draft an email).

Response 202 :

```
{ "approval_id": "a8", "message": "Draft queued for approval" }
```

POST /findings/{id}/dismiss

Dismiss a finding.

Response 200 :

```
{ "id": "f1", "dismissed": true }
```

9. Chat

GET /chat/history

Returns the conversation history for the authenticated user.

Query params:

Param	Type	Description
limit	int	Default: 50
before_id	string	Cursor for pagination

Response 200 :

```
{
  "messages": [
    {
      "id": "msg1",
      "role": "user",
      "content": "Find SMSF clients whose annual returns are overdue...",
      "timestamp": "2026-05-23T13:00:00+10:00"
    },
    {
      "id": "msg2",
      "role": "assistant",
      "content": "I checked XPM and found 4 SMSF clients...",
      "timestamp": "2026-05-23T13:00:12+10:00"
    }
  ]
}
```

POST /chat/message

Send a new chat message and get a streaming response.

Request body:

```
{
  "content": "Which BAS clients haven't sent figures yet for Q4?"
}
```

Response: Server-Sent Events (SSE) stream, Content-Type: text/event-stream

Each event:

```
data: {"type": "delta", "content": "I checked XPM and found"}
data: {"type": "delta", "content": " 3 clients..."}
data: {"type": "done", "message_id": "msg3", "tokens": 842}
```

The frontend appends `delta` content to the assistant message bubble in real time, then finalises on `done`.

10. Specialists

GET /specialists

Returns all specialist agent definitions.

Response 200 :

```
{
  "items": [
    {
      "id": "s1",
      "name": "GST Specialist",
      "description": "Expert in GST, BAS, input tax credits, and GST registration.",
      "version": "v4",
      "last_updated": "2026-05-01",
      "status": "active",
      "runs_this_month": 34
    }
  ]
}
```

status values: "active" | "draft" | "archived"

GET /specialists/{id}/prompt

Returns the active system prompt for a specialist.

Response 200 :

```
{
  "id": "s1",
  "version": "v4",
  "prompt": "You are the GST Specialist for MC & S Accountants...",
  "test_cases": [
    {
      "case": "GST on mixed supply – residential + commercial",
      "verdict": "pass",
      "model": "claude-sonnet"
    }
  ]
}
```

PUT /specialists/{id}/prompt

Update a specialist's system prompt.

Request body:

```
{
  "prompt": "You are the GST Specialist for MC & S Accountants...\n[updated content]"
}
```

Response 200 :

```
{ "id": "s1", "version": "v5", "updated_at": "2026-05-23T15:00:00+10:00" }
```

11. Settings

GET /settings/firm

Returns firm-level settings.

Response 200 :

```
{
  "firm_name": "MC & S Accountants",
  "abn": "12 345 678 901",
  "timezone": "Australia/Melbourne",
  "primary_email_domain": "mcands.com.au",
  "shadow_mode": true,
  "approval_threshold": 0.85
}
```

PATCH /settings/firm

Update firm settings. Send only the fields to change.

Request body:

```
{
  "shadow_mode": false,
  "approval_threshold": 0.90
}
```

Response 200 : Updated firm settings object (same shape as GET).

GET /settings/user

Returns the authenticated user's profile and learned style.

Response 200 :

```
{
  "id": "u1",
  "name": "Eliza Marchetti",
  "email": "eliza@mcands.com.au",
  "role": "Senior Accountant",
  "style_profile": {
    "tone": "Warm-formal",
    "salutation": "Hi {first name}",
    "sign_off": "Kind regards",
    "language": "Australian English",
    "characteristic_phrase": "Happy to chat through this"
  }
}
```

PATCH /settings/user

Update user profile fields.

Request body:

```
{
  "name": "Eliza Marchetti",
  "role": "Senior Accountant"
}
```

GET /settings/token-usage

Returns token consumption breakdown for the current month.

Response 200 :

```
{
  "period": "2026-05",
  "used": 18245,
  "cap": 50000,
  "by_plugin": [
    { "plugin": "Smart Responder", "tokens": 8420, "pct": 46 },
    { "plugin": "BAS Reminder", "tokens": 4210, "pct": 23 },
    { "plugin": "Morning Briefing", "tokens": 3210, "pct": 18 },
    { "plugin": "Other plugins", "tokens": 2405, "pct": 13 }
  ]
}
```

12. WebSocket Events

Connect at: `wss://coworker.mcands.com.au/ws/{user_id}`

The frontend uses the WebSocket connection to update the Dashboard live activity feed and the Approvals badge count in real time. All messages are JSON.

Events the server sends to the frontend

activity.new

A new agent trace has been created.

```
{
  "type": "activity.new",
  "data": {
    "id": "tr12",
    "time_display": "14:45",
    "timestamp": "2026-05-23T14:45:00+10:00",
    "plugin": "Smart Responder",
    "client": "Henderson & Co",
    "action": "Drafted reply to \"Q4 BAS preparation\"",
    "status": "queued",
    "tokens": 1204,
    "duration": "11.2s"
  }
}
```

approval.new

A new approval item has been queued.

```
{
  "type": "approval.new",
  "data": {
    "id": "a8",
    "client": "Henderson & Co",
    "plugin": "Smart Responder",
    "subject": "Q4 BAS preparation",
    "confidence": 0.81,
    "sla_risk": "low",
    "category": "email_draft",
    "created_at": "2026-05-23T14:45:00+10:00"
  }
}
```

approval.resolved

An approval has been approved or rejected (by any user in the firm).

```
{
  "type": "approval.resolved",
  "data": {
    "id": "a1",
    "status": "approved",
    "resolved_by": "Eliza Marchetti",
    "resolved_at": "2026-05-23T14:31:00+10:00"
  }
}
```

finding.new

A new proactive finding has been generated.

```
{
  "type": "finding.new",
  "data": {
    "id": "f5",
    "severity": "medium",
    "category": "LIFECYCLE",
    "title": "Henderson & Co hasn't been contacted in 91 days",
    "created_at": "2026-05-23T14:45:00+10:00"
  }
}
```

plugin.status

A plugin's status has changed (e.g. started running, completed, errored).

```
{
  "type": "plugin.status",
  "data": {
    "id": "p1",
    "status": "running",
    "run_id": "run_abc123"
  }
}
```

Events the frontend sends to the server

`ping`

Keepalive. Send every 30 seconds.

```
{ "type": "ping" }
```

Server responds with:

```
{ "type": "pong" }
```

13. Shared Types

Approval status flow

```
pending → approved → sent  
    ↘ rejected
```

Activity status values

Value	Meaning
<code>queued</code>	Awaiting human approval
<code>approved</code>	Approved, pending send
<code>sent</code>	Dispatched to recipient
<code>rejected</code>	Rejected by human
<code>running</code>	Currently executing
<code>error</code>	Failed with error

SLA risk thresholds

The frontend colours the SLA display based on `sla_risk`:

- `low` → green `#16a34a`
- `medium` → orange `#eb881f`
- `high` → red `#e11d48`

The backend should calculate `sla_risk` based on the plugin's configured SLA window and time elapsed.

Pagination envelope

All list endpoints return:

```
{
  "total": 50,
  "items": [ ... ]
}
```

Error envelope

All error responses:

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "Approval a99 not found"
  }
}
```

14. Frontend Wiring Guide

When you're ready to connect the backend, here is the exact change to make in the frontend codebase:

File to edit: `src/data/mock.ts` → replace with `src/api/client.ts`

Create `src/api/client.ts`:

```

const BASE = '/api/v1'

function getToken() {
  return localStorage.getItem('access_token')
}

async function get<T>(path: string, params?: Record<string, string>):
Promise<T> {
  const url = new URL(BASE + path, window.location.origin)
  if (params) Object.entries(params).forEach(([k, v]) =>
url.searchParams.set(k, v))
  const res = await fetch(url.toString(), {
    headers: { Authorization: `Bearer ${getToken()}` },
  })
  if (!res.ok) throw new Error(`GET ${path} → ${res.status}`)
  return res.json()
}

async function post<T>(path: string, body?: unknown): Promise<T> {
  const res = await fetch(BASE + path, {
    method: 'POST',
    headers: {
      Authorization: `Bearer ${getToken()}`,
      'Content-Type': 'application/json',
    },
    body: body ? JSON.stringify(body) : undefined,
  })
  if (!res.ok) throw new Error(`POST ${path} → ${res.status}`)
  return res.json()
}

async function patch<T>(path: string, body: unknown): Promise<T> {
  const res = await fetch(BASE + path, {
    method: 'PATCH',
    headers: {
      Authorization: `Bearer ${getToken()}`,
      'Content-Type': 'application/json',
    },
    body: JSON.stringify(body),
  })
  if (!res.ok) throw new Error(`PATCH ${path} → ${res.status}`)
  return res.json()
}

export const api = {

```

```

dashboard: {
  summary: () => get('/dashboard/summary'),
},
approvals: {
  list: () => get('/approvals'),
  approve: (id: string, editedDraft?: string) =>
    post(`/approvals/${id}/approve`, { edited_draft: editedDraft }),
  reject: (id: string, reason?: string) =>
    post(`/approvals/${id}/reject`, { reason }),
},
plugins: {
  list: () => get('/plugins'),
  toggle: (id: string, enabled: boolean) => patch(`/plugins/${id}`, {
enabled }),
  run: (id: string) => post(`/plugins/${id}/run`),
},
memory: {
  list: (q?: string, type?: string) =>
    get('/memory', { ...(q && { q }), ...(type && type !== 'all' && { type
})) }),
},
knowledgeGraph: {
  nodes: (clientId?: string) =>
    get('/knowledge-graph/nodes', clientId ? { client_id: clientId } :
undefined),
},
activity: {
  list: () => get('/activity'),
},
findings: {
  list: () => get('/findings'),
  act: (id: string) => post(`/findings/${id}/act`),
  dismiss: (id: string) => post(`/findings/${id}/dismiss`),
},
chat: {
  history: () => get('/chat/history'),
  send: (content: string) => post('/chat/message', { content }),
},
specialists: {
  list: () => get('/specialists'),
  prompt: (id: string) => get(`/specialists/${id}/prompt`),
  updatePrompt: (id: string, prompt: string) =>
    patch(`/specialists/${id}/prompt`, { prompt }),
},
settings: {
  firm: () => get('/settings/firm'),
}

```

```

    updateFirm: (data: Record<string, unknown>) => patch('/settings/firm',
data),
    user: () => get('/settings/user'),
    updateUser: (data: Record<string, unknown>) => patch('/settings/user',
data),
    tokenUsage: () => get('/settings/token-usage'),
  },
}

```

Then in each page component, replace the mock import:

```

// Before
import { mockApprovals } from '../data/mock'

// After
import { useEffect, useState } from 'react'
import { api } from '../api/client'

const [approvals, setApprovals] = useState([])
useEffect(() => { api.approvals.list().then(r => setApprovals(r.items)) },
[])

```

WebSocket connection

Add to `src/App.tsx` (or a `useWebSocket` hook):

```
useEffect(() => {
  const userId = getCurrentUserId() // from JWT
  const ws = new WebSocket(`wss://coworker.mcands.com.au/ws/${userId}`)
  ws.onmessage = (e) => {
    const event = JSON.parse(e.data)
    if (event.type === 'approval.new') {
      // increment badge, add to approvals list
    }
    if (event.type === 'activity.new') {
      // prepend to activity feed
    }
    if (event.type === 'finding.new') {
      // increment findings badge
    }
  }
  const ping = setInterval(() => ws.send(JSON.stringify({ type: 'ping' })),
30000)
  return () => { clearInterval(ping); ws.close() }
}, [])
```

Document generated from `src/data/mock.ts` and all 10 page components. Last updated: 2026-05-23.